

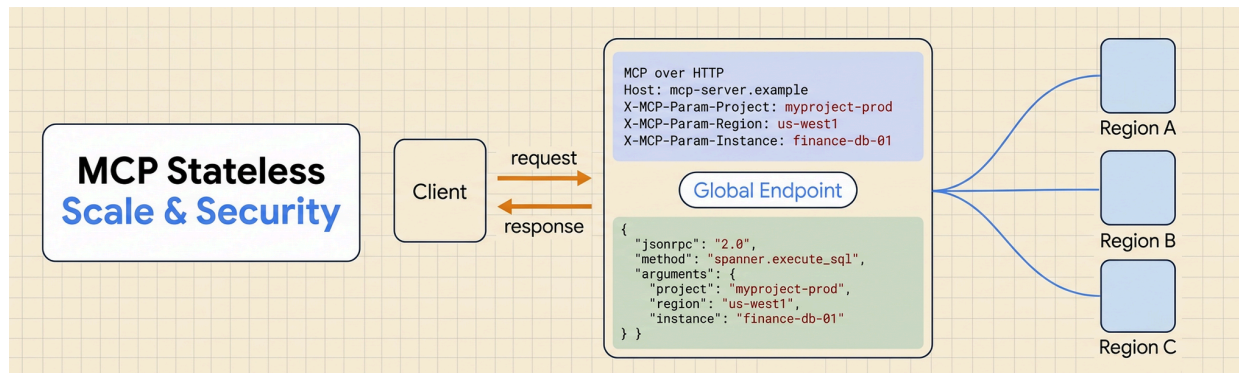
<https://developers.googleblog.com/scaling-ai-agent-infrastructure-with-the-mcp-stateless-updates/>

Scaling AI Agent Infrastructure with the MCP Stateless updates

AUG. 5, 2026

[Kurtis Van Gent](#) Senior Staff Software Engineer Google Cloud Data

[Alan Blount](#) Technical Product Manager Google Cloud AI



As you deploy agentic workflows and scale up your users, your bottlenecks change. When the Model Context Protocol (MCP) was first introduced in late 2024, it provided an elegant, session-oriented framework that allowed LLMs to negotiate capabilities, invoke external tools, and retrieve contextual resources. It was perfect for a single client talking to a single server on a local machine and optimized for stdio.

But when we at Google began deploying MCP servers across our cloud-native infrastructure, we hit a hard wall. The original protocol-level session model required persistent state, handshakes, and session pinning. In short, it was built on stateful transports that broke the core tenets of modern cloud-native scalability.

To solve this, [Google led the charge](#) to decouple the protocol from stateful transport constraints. Our teams needed MCP to scale across millions of concurrent queries on Google Cloud, and we knew that you all needed MCP to be ready for real-world enterprise scale too. Working closely with Hugging Face and other industry partners, we co-founded the MCP Transports Working Group.

Today, we are thrilled to celebrate the culmination of that work: **the 2026-07-28 Model Context Protocol specification release candidate**, which is [already being widely adopted](#). This landmark release removes transport-level session management entirely, giving you a stateless protocol core that scales on ordinary HTTP load-balanced infrastructure.

It's the biggest change to MCP spec since launch, and if you don't read the rest of this article, rest assured, it's a change for the better - **more scale, more secure, just as easy**.

Why Sessions Were a Production Bottleneck

In the original protocol model (specification version 2025-11-25) [392], connecting to an MCP server over HTTP required a stateful initialization process:

```
// POST /mcp - Legacy 2025-11-25 Handshake { "jsonrpc": "2.0", "id": 1,  
"method": "initialize", "params": { "protocolVersion": "2025-11-25",  
"capabilities": {}, "clientInfo": { "name": "my-app", "version": "1.0" } } }
```

JSON

Copied

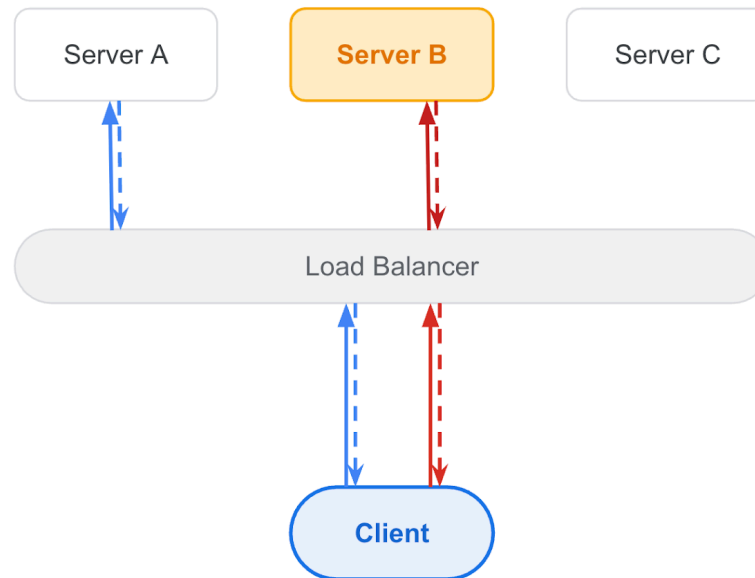
The server responded with an `Mcp-Session-Id` header. To make any subsequent tool call or resource query, the client had to include that unique session ID on every request, pinning the client to the specific container or pod that held its in-memory session state.

This stateful constraint breaks the horizontal scaling models that cloud-native engineers depend on:

- **The Load Balancing Tax:** Standard round-robin load balancers do not know which container holds which in-memory session. Deploying behind a Kubernetes cluster with three pods meant a second request from a client would randomly hit another pod, returning a `400 Session Not Found` error.

- **Sticky Routing Overheads:** Developers were forced to configure sticky session affinity rules at the load balancer level, which prevents even distribution of traffic and makes autoscaling highly inefficient.
- **Zero Fault Tolerance:** If a pod restarts or crashes, the session state is instantly lost, throwing transient errors back to active client chats and ruining the user experience.
- **Complex Infrastructure Demands:** Running remote MCP servers required shared Redis session stores or complex gateway-level packet inspection, introducing massive latency and operational costs.

Multi-Round Trip Request Problem



Request Lifecycle

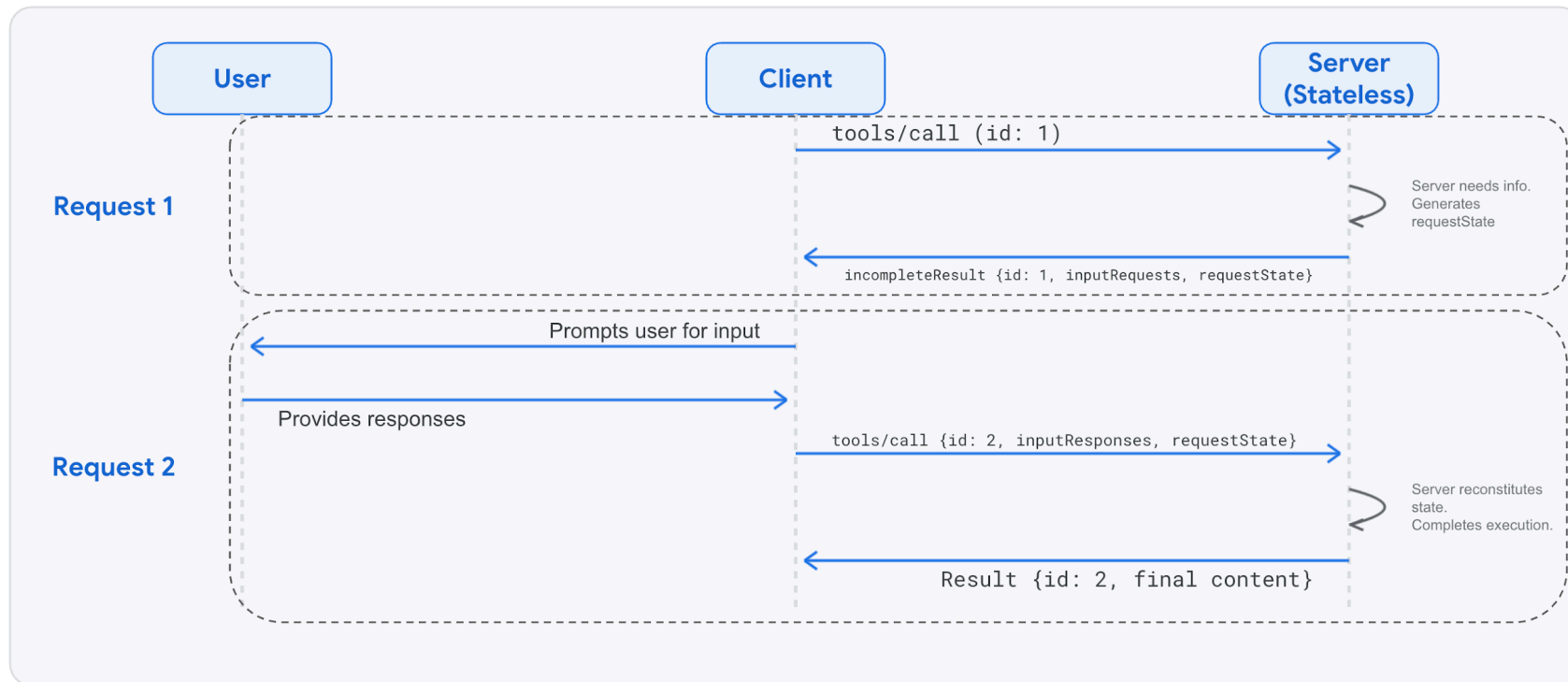
1. **Initial Call:** Client starts a tool call. LB routes to **Server A**.
2. **Elicitation:** Server A sends back an elicitation request (on SSE Stream).
3. **The Problem:** Response to elicitation is a **new** request. Load Balancer routes it to **Server B**, which lacks the context.

The New Request Model: Going Fully Stateless

The new 2026-07-28 specification solves this by making the protocol core completely stateless. **The handshake is gone.** The `initialize / initialized` handshake (SEP-2575) and the logical `Mcp-Session-Id` header (SEP-2567) have been removed entirely.

Instead, **every request is now self-describing and independent.** Protocol version, client info, and client capabilities that used to be exchanged once at connection setup now travel in a `_meta` field inline on every single request.

SEP-2322: Multi Round-Trip Requests



Here is how a stateless tool call looks under the new 2026-07-28 specification:

```
POST /mcp HTTP/1.1 Host: mcp-server.example MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call Mcp-Name: search Content-Type: application/json {
  "jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": { "name":
    "search", "arguments": { "q": "otters" }, "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientCapabilities": {},
      "io.modelcontextprotocol/clientInfo": { "name": "my-app", "version": "1.0" } }
    } }
```

Plain text

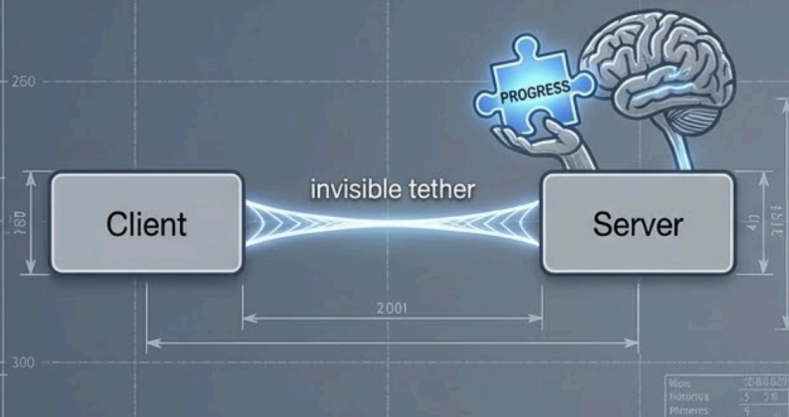
Architectural Advantages of Stateless Core

- **Standard Round-Robin Routing:** Because any container instance can handle any incoming request, you can throw your stateful MCP servers behind a plain round-robin load balancer.
- **Seamless Serverless Deployment:** You can now run MCP servers as serverless functions on platforms like Google Cloud Run or Google Cloud Functions. Since there is no persistent connection to maintain, your servers spin down to zero when idle, drastically reducing costs.

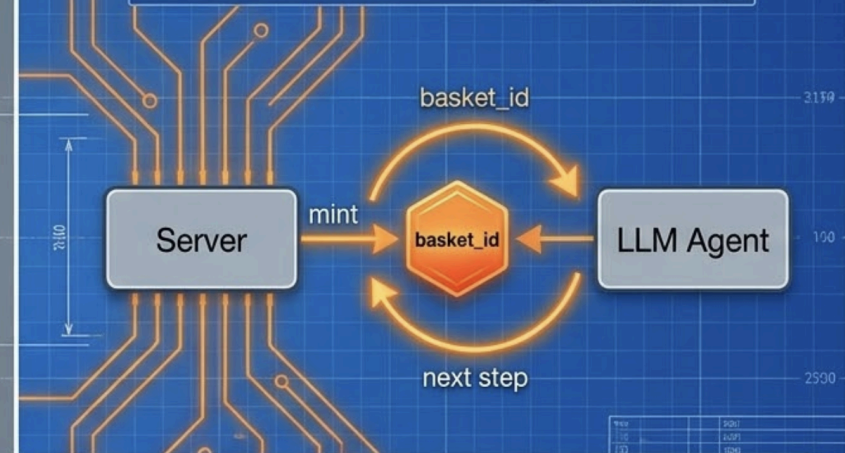
- **Transparent Failover:** Pod restarts, rollouts, and autoscaling events are completely invisible to the client. If a container crashes, the load balancer routes the very next request to a healthy peer with zero session disruption.
- **No Redis Sessions Needed:** Major production servers, such as the GitHub MCP Server, have already upgraded to this spec and completely removed Redis session storage, eliminating database writes and reads on every single call to make interactions snappier.

From Implicit Memory to **Explicit State Handles**

The **Legacy Bottleneck** (Implicit)



The **Cloud-Native Edge** (Explicit)



Implicit Memory (Deprecated): Server tracks progress in-memory via Mcp-Session-Id.

Explicit State Handles (Now): Server generates a durable handle. The LLM holds it and passes it back as a tool argument.

Multi-Round-Trip Requests (MRTR): Tools can now return `InputRequiredResult`. The server pauses, asks the user for input, and the client retries the request with answers—zero blocked connections.

HTTP Standardization: Routable, Cacheable, and Traceable

Without protocol sessions, we needed standard mechanisms to route and govern traffic efficiently. Working under the Transports Working Group, we helped design **SEP-2243 (HTTP Standardization)** [302, 542].

Streamable HTTP POST requests now carry specific HTTP headers:

- `Mcp-Protocol-Version`: The version of the protocol.
- `Mcp-Method`: The JSON-RPC method being executed (such as `tools/call`).
- `Mcp-Name`: The specific tool, prompt, or resource name being invoked.

These headers are mirrored to match the JSON-RPC body. If they disagree, the server rejects the request with a `-32020` header mismatch code.

No More Deep Packet Inspection

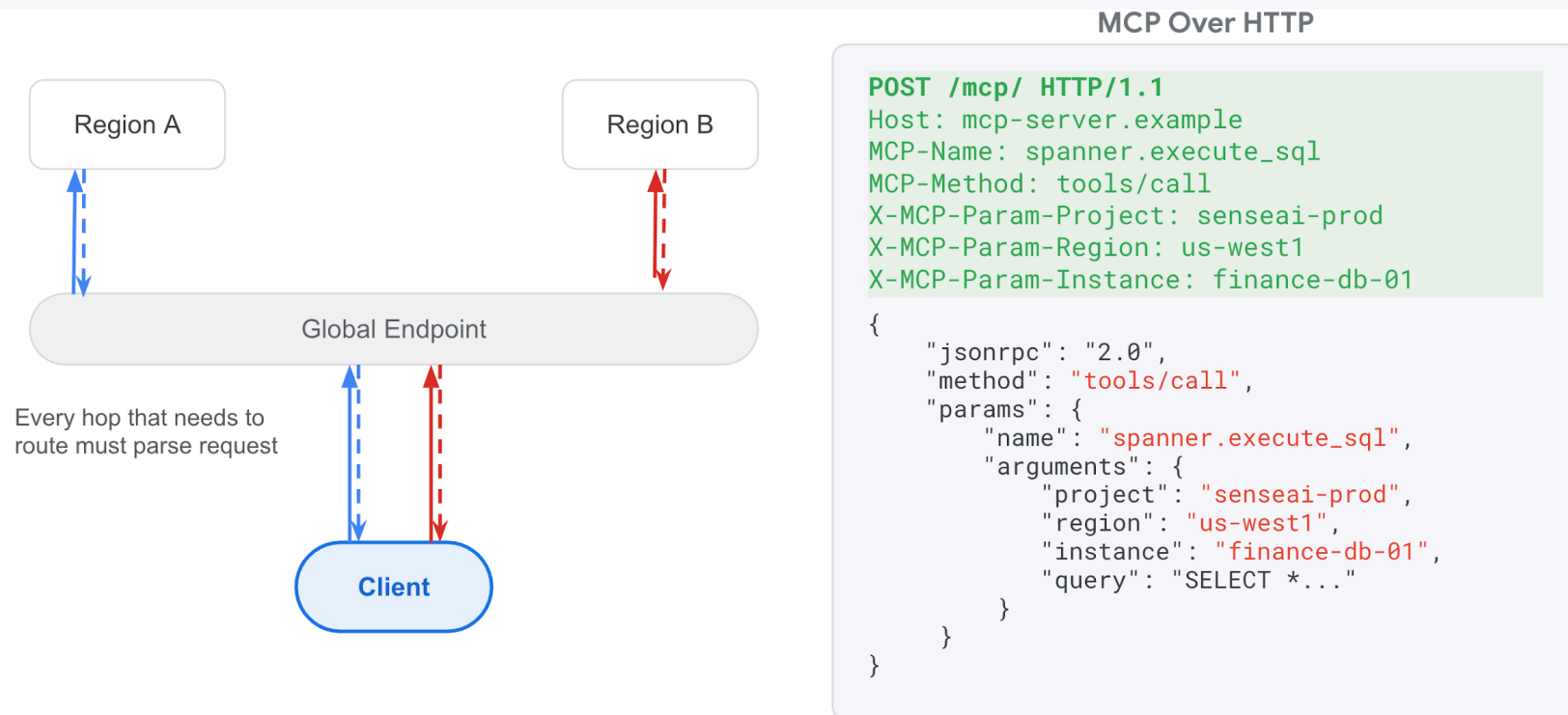
By promoting these values to standard HTTP headers, proxies, gateways, and load balancers can route, rate-limit, and audit traffic **without inspecting the request body**. For security and logging teams, this is a massive win that drastically lowers the latency and processing overhead at the gateway layer.

Intelligent Caching with ttlMs (SEP-2549)

To eliminate the need for long-lived Server-Sent Events (SSE) connections just to monitor if a tool or prompt list changed, the spec introduces caching fields modeled after HTTP's Cache-Control. Tool and resource results can now return a `ttlMs` (Time-to-Live in

milliseconds) and a `cacheScope`. Clients know exactly how long a `tools/list` response is fresh and whether it is safe to cache across multiple users.

SEP-2243: HTTP Standardization



Multi Round-Trip Requests (MRTR): Handling Elicitations Statelessly

One of the most complex challenges we faced in a stateless world was how to handle server-to-client requests. Under previous versions, if an MCP server needed user clarification (an "elicitation prompt") or a confirmation during a tool call, it had to keep an SSE connection open to push that request to the client.

Multi Round-Trip Requests (SEP-2322) solves this problem beautifully by restructuring the interaction lifecycle into self-contained steps:

Instead of blocking the thread or holding a connection open, the server immediately returns an `InputRequiredResult` with a `requestState` payload containing serialized context [398]:

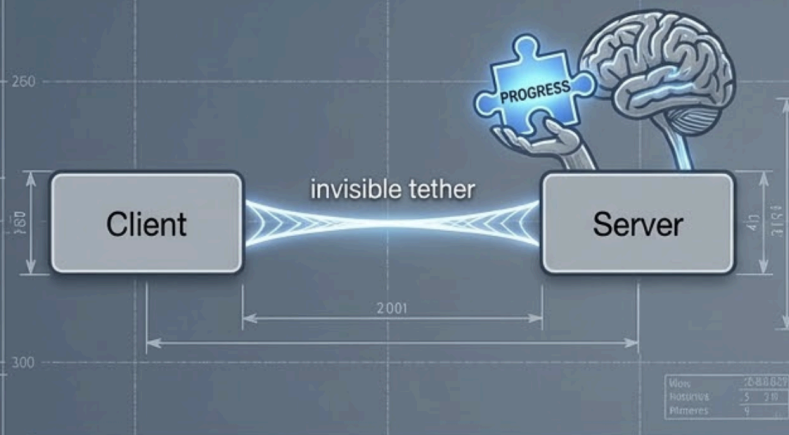
```
// InputRequiredResult Returned from Server { "resultType": "inputRequired",  
"inputRequests": { "confirm": { "type": "elicitation", "message": "Are you sure  
you want to delete these 3 files?", "schema": { "type": "boolean" } } },  
"requestState": "eyJzdGVwIjoxLCJmaWxlcyI6WyJhIiwiaYiIsImMiXX0=" }
```

JSON

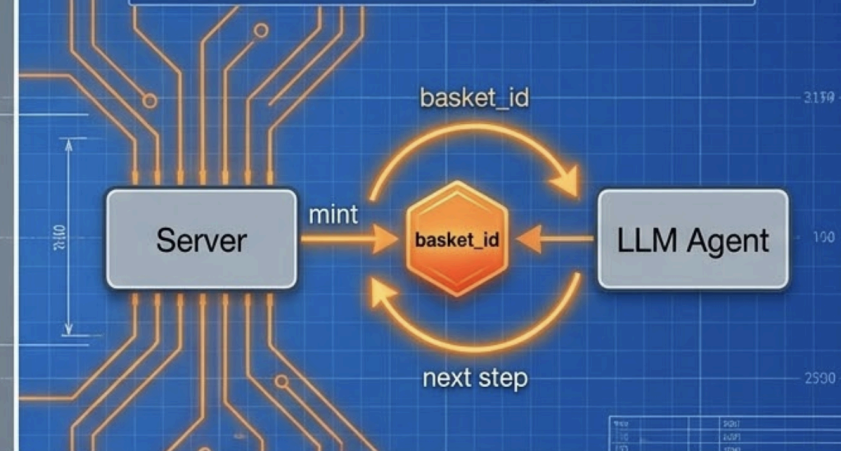
The client prompts the user, gathers the boolean answer, and reissues the call with `inputResponses` and the echoed `requestState`. Because the `requestState` contains everything needed to resume the task, **any server instance behind your load balancer can pick up the retry request!**

From Implicit Memory to **Explicit State Handles**

The **Legacy Bottleneck** (Implicit)



The **Cloud-Native Edge** (Explicit)



Implicit Memory (Deprecated): Server tracks progress in-memory via Mcp-Session-Id.

Explicit State Handles (Now): Server generates a durable handle. The LLM holds it and passes it back as a tool argument.

Multi-Round-Trip Requests (MRTR): Tools can now return `InputRequiredResult`. The server pauses, asks the user for input, and the client retries the request with answers—zero blocked connections.

The Tasks Extension (SEP-2663): Async Work Without Blocking

Sometimes a tool call simply takes a long time to run. A database backup, a CRM sync, or a refund through a payment gateway can take anywhere from 10 to 60 seconds. Holding the client connection open blocks the customer conversation and creates massive connection queues.

The **Tasks Extension** graduates from an experimental feature to a robust, first-class protocol extension. Now, when a client calls a long-running tool, the server immediately returns a `taskId` and kicks off the execution in the background:

```
// Example: Kicking off an async task in a TypeScript server
server.tool(
  "process_refund", { orderId: z.string(), amount: z.number() }, async ({
    orderId, amount }) => { const taskId = randomUUID(); // Store initial task
    state in a shared datastore (e.g. Redis) await setTaskState(taskId, { status:
    "working" }); // Process the refund asynchronously in the background
    processRefundAsync(taskId, orderId, amount); // Return immediately to keep the
```

```
conversation flowing return { content: [ { type: "text", text: JSON.stringify({  
taskId, status: "working", message: `Refund of ${amount} for order ${orderId}  
is processing. Task ID: ${taskId}` }) } ] }; } );
```

JavaScript

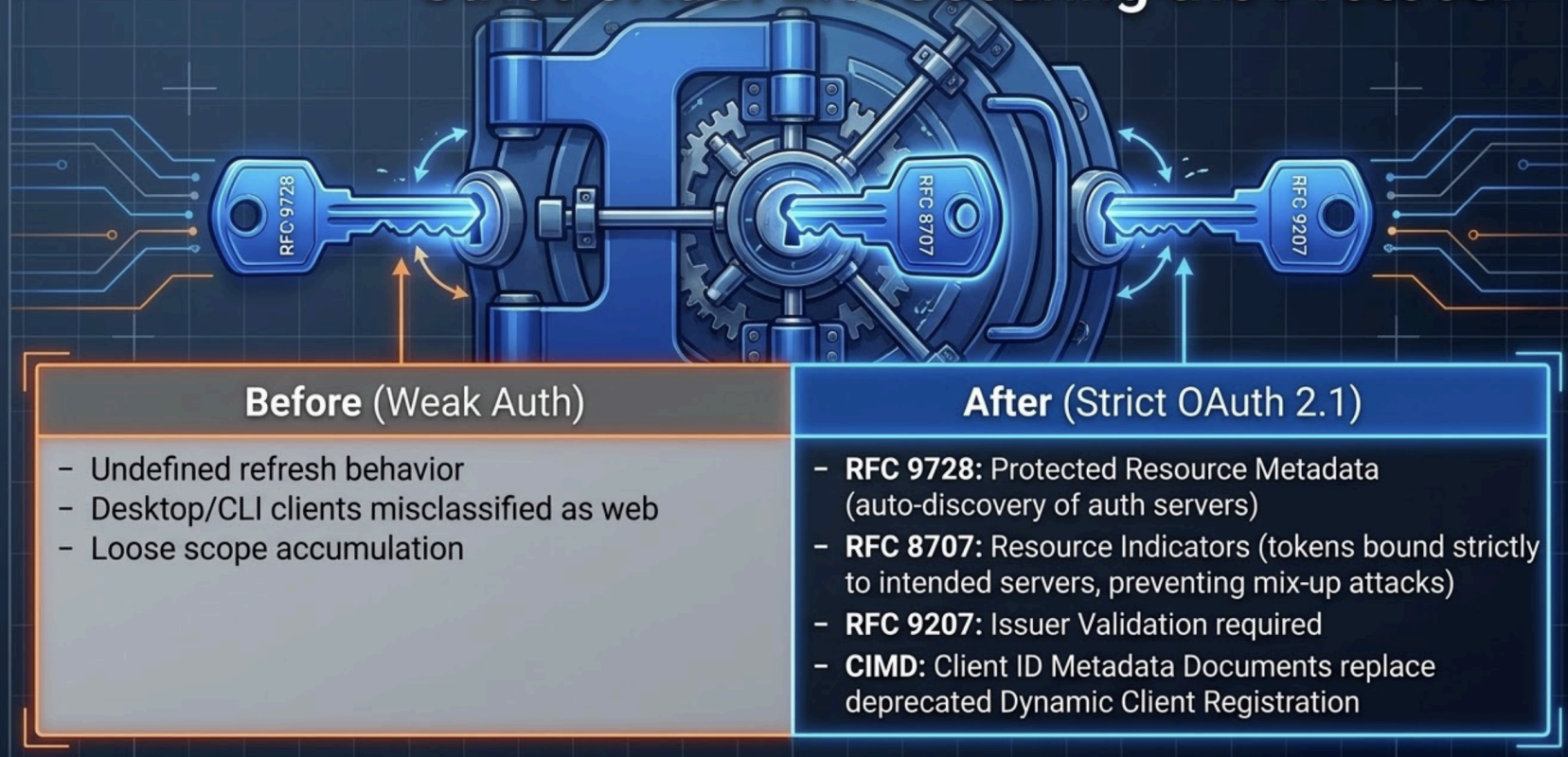
The client continues the conversation, telling the user their request is processing, and can poll or subscribe using standard `tasks/get` and `tasks/update` primitives to monitor progress and fetch the final results.

Clear Security & Capability Boundaries

As the responsibility of managing state shifts from the transport layer to the application layer, security becomes paramount. The 2026-07-28 spec delivers several crucial security enhancements:

- **Issuer Verification (RFC 9207):** Public clients must validate the `iss` parameter on authorization responses, protecting against session hijacking and redirect-based attacks in multi-server architectures.
- **Resource Indicators (RFC 8707):** Clients explicitly specify which MCP server a token is intended for, solving the "confused deputy" delegation problem.
- **Full JSON Schema 2020-12 for Tools:** Input schemas can now use advanced composition structures (such as `oneOf`, `anyOf`, `allOf`) and local `$ref` definitions, making parameters highly descriptive and strictly validated.

Strict OAuth 2.1: Securing the Protocol



Deprecations and a Predictable Future (SEP-2577)

For the first time, MCP now has a formal deprecation policy. Features move through a structured *Active* -> *Deprecated* -> *Removed* lifecycle with a **minimum 12-month transition window**. Three features enter deprecation today:

1. **Roots:** Replaced by explicit tool parameters, resource URIs, or server configuration.
2. **Sampling:** Replaced by calling LLM provider APIs directly.
3. **Logging:** Replaced by standard `stderr` for `stdio` connections, or OpenTelemetry for structured cloud observability.

Getting Started and Migrating

All four Tier-1 SDKs (TypeScript, Python, Go, and C#) already have beta releases available supporting the 2026-07-28 specification. We highly encourage you to start testing these in your staging environments today.

Python (mcp v2)

In Python, the `MCPServer` decorator API is fully compatible [303]. You can install the beta directly with:

```
pip install "mcp[cli]==2.0.0b1"
```

Shell

TypeScript (split packages)

TypeScript v2 replaces the monolithic `@modelcontextprotocol/sdk` package with modular, focused libraries to keep your dependencies light. Install them with:

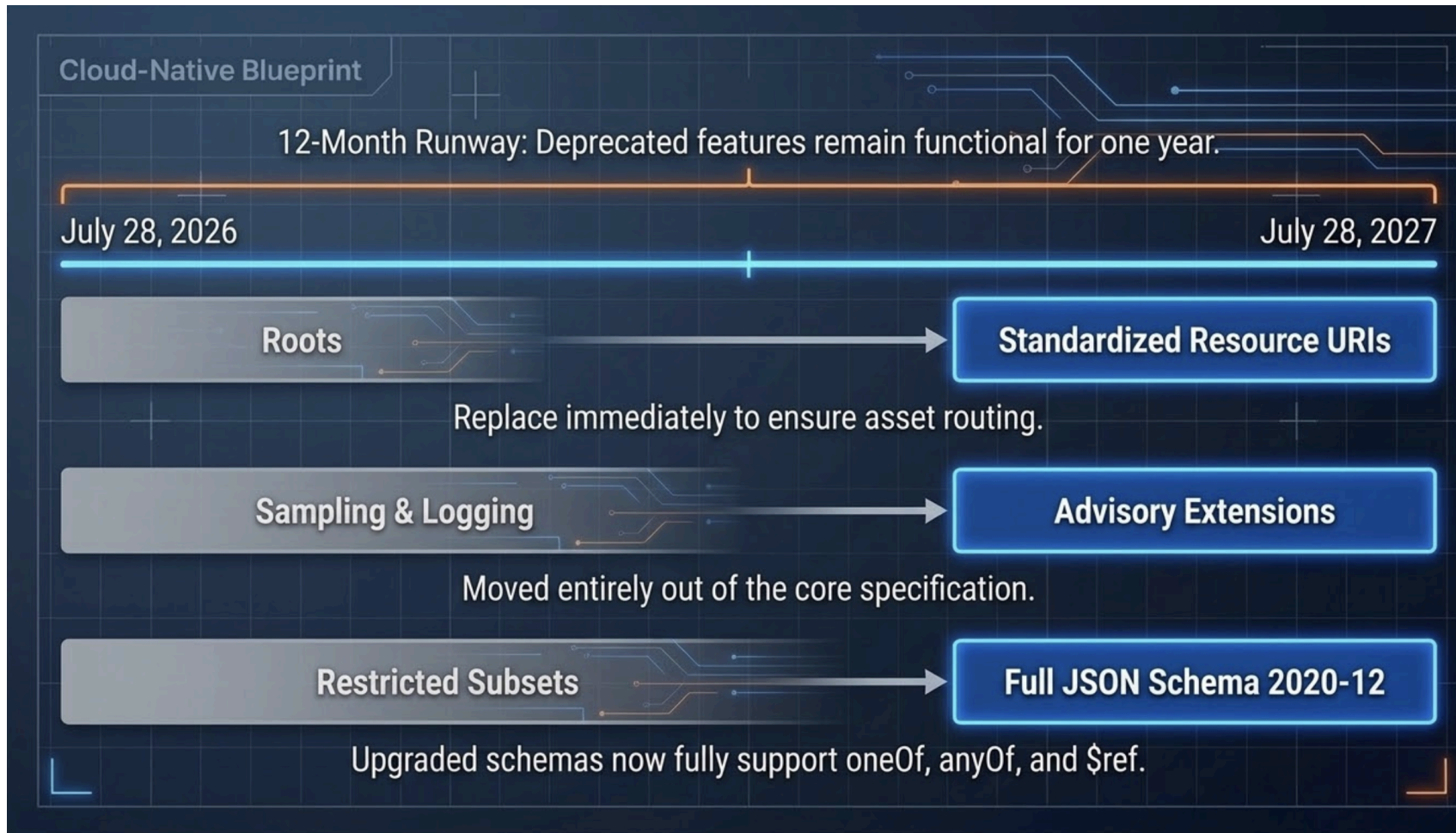
```
npm install @modelcontextprotocol/server@beta npm install  
@modelcontextprotocol/client@beta
```

Shell

A convenient codemod is available to handle standard API renames (like renaming `.tool()` to `registerTool`):

```
npx @modelcontextprotocol/codemod@beta v1-to-v2 .
```

Shell



Conclusion

The 2026-07-28 specification marks a watershed moment for the Model Context Protocol, transitioning it from a promising local integration layer into the foundational, open infrastructure for enterprise AI applications.

Thank you to the huge effort from all of the [MCP Transports Working Group](#) and other teams who worked to make this happen, from across many companies. Thanks also to the Google team who maintain the [Go MCP SDK](#) and shipped [v1.7.0](#) on July 28th which was ready on launch day and powers major integrations like [Github MCP Server](#).

Google's push for stateless transports was born out of necessity. We needed a protocol robust enough to handle the massive scale of our global developers, and we wanted to ensure that every developer, whether building on Google Cloud or anywhere else, had access to highly reliable, secure, and infinitely scalable agentic infrastructure.

By decoupling state from the transport layer, we have made load balancing boring, autoscaling seamless, and serverless deployment a reality. We can't wait to see the incredibly scalable AI agents you build on top of this new stateless foundation!

Explore more:

- Read the [full 2026-07-28 Specification](#).
- Check out the [TypeScript SDK Upgrading Guide](#).
- Star and contribute to the [Go SDK and Examples](#).